

LINQ Empty Method in C# with Examples

In this article, I will discuss the **LINQ Empty Method in C#** with Examples. Please read our previous article discussing the [LINQ Repeat Method in C#](#) with Examples. Like the LINQ Range and Repeat Methods, the Empty method also belongs to the Generation Operator Category. As we already discussed, Generation Operators allow us to create some specific type of Array, Sequence, or Collection using a single expression instead of creating and populating them manually using some loop.

LINQ Empty Method in C#:

The Empty method in LINQ generates an empty sequence of a particular type. This method is useful when you need to return an empty sequence from a method without returning null or when you need to start with an empty sequence and conditionally populate it with elements based on further logic.

The **LINQ Empty Method in C#** is a static method included in the static Enumerable class. The Empty Method returns an empty collection (i.e., IEnumerable<T>) of a specified type. The following is the signature of this method.

```
public static IEnumerable<TResult> Empty<TResult>();
```

Here, **TResult** specifies the type parameter of the returned generic IEnumerable<TResult>. This method returns an empty IEnumerable<TResult> whose type argument is TResult.

Example to Understand Empty Method in C#:

Let us see an example of the LINQ Empty Method in C#. In the following example, we have created two empty collections using the LINQ Empty Method. The first Empty method returns an empty collection of strings, while the second Empty method returns an empty collection of students.

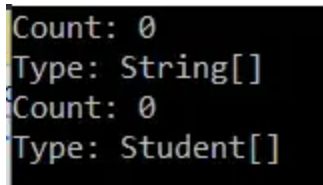
```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Empty Collection of Strings
            IEnumerable<string> emptyCollection1 = Enumerable.Empty<string>();
            //Creating Empty Collection of Student
            IEnumerable<Student> emptyCollection2 = Enumerable.Empty<Student>();
            //Printing the Type and Count of emptyCollection1
            Console.WriteLine("Count: {0} ", emptyCollection1.Count());
            Console.WriteLine("Type: {0} ", emptyCollection1.GetType().Name);
            //Printing the Type and Count of emptyCollection2
            Console.WriteLine("Count: {0} ", emptyCollection2.Count());
        }
    }
}
```

```

        Console.WriteLine("Type: {0} ", emptyCollection2.GetType().Name);
        Console.ReadKey();
    }
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
}
}

```

Output:



```

Count: 0
Type: String[]
Count: 0
Type: Student[]

```

Why do we need the Empty Method in C#?

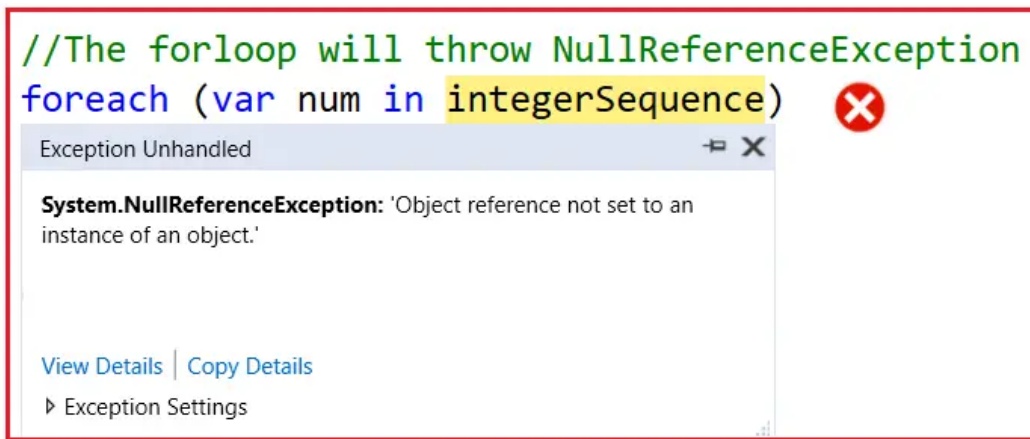
Let us understand the need for the LINQ Empty Method in C# with an example. Consider one scenario where our application calls a method that returns an **IEnumerable<int>**. There might be a situation where the method returns null. In the following example, the **GetData()** method is returning null. So, when we run the following program, it will throw a NULL Reference Exception.

```

using System.Collections.Generic;
using System;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //GetData() Method Returning Null
            IEnumerable<int> integerSequence = GetData();
            //The forloop will throw NullReferenceException
            foreach (var num in integerSequence)
            {
                Console.WriteLine(num);
            }
            Console.ReadKey();
        }
        private static IEnumerable<int> GetData()
        {
            return null;
        }
    }
}

```

So, when you run the above code, you will get the following Runtime Exception.



The above problem can be fixed in two ways.

Solution1: Checking Null Before using inside the Loop

We need to check for **NULL** before looping through the items in the collection, as shown in the below example.

```
using System.Collections.Generic;
using System;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            IEnumerable<int> integerSequence = GetData();
            if(integerSequence != null)
            {
                foreach (var num in integerSequence)
                {
                    Console.WriteLine(num);
                }
            }
            Console.ReadKey();
        }
        private static IEnumerable<int> GetData()
        {
            return null;
        }
    }
}
```

Solution2: Using the Empty Method

We can solve the problem using the LINQ Empty Method, as shown in the example below. Here, we use the **NULL-COALESCEING** operator, which checks if the **GetData()** method returns **NULL**, then initializes the **integerSequence** variable with an empty **IEnumerable<int>**.

```
using System.Linq;
using System.Collections.Generic;
using System;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            IEnumerable<int> integerSequence = GetData() ?? Enumerable.Empty<int>();
            foreach (var num in integerSequence)
            {
                Console.WriteLine(num);
            }
            Console.ReadKey();
        }
        private static IEnumerable<int> GetData()
        {
            return null;
        }
    }
}
```

While the Empty method is not as commonly used as other LINQ methods like Where, Select, or OrderBy, it can be useful in scenarios where you need to initialize sequences or collections without any initial data but of a specific type.

When to Use the LINQ Empty() Method in C#?

The `Enumerable.Empty<T>` method in C# LINQ is typically used when creating an empty sequence or collection of a specific type. It is useful when you require an initial empty collection to work with or as a placeholder. Here are some common situations where you might use the `Enumerable.Empty<T>` method:

Initializing an Empty Sequence:

When you need to start with an empty sequence of a particular type before populating it with data dynamically, this is common when building LINQ queries or initializing collections for later use.

Use Case: When you need to initialize a variable to an empty sequence to ensure that subsequent LINQ queries or operations do not throw a `NullReferenceException`.

Example: `IEnumerable<int> numbers = Enumerable.Empty<int>();`

Placeholder for Default Values:

In some cases, you may need to provide a default value for an operation or return a known empty sequence as a result. `Enumerable.Empty<T>` can be used for this purpose.

`var result = GetResults() ?? Enumerable.Empty<ResultType>();`

Testing for Empty Collections: You can use `Enumerable.Empty<T>` when writing unit tests or validations to ensure your code handles empty collections correctly. It allows you to create empty collections for testing easily.

```
var emptyCollection = Enumerable.Empty<string>();  
// Perform tests with the empty collection
```

Providing Default Data:

When working with methods that require input data but can also accept empty data, you can use `Enumerable.Empty<T>` as a default value to represent the absence of data.

```
var data = GetData() ?? Enumerable.Empty<DataItem>();
```

Initial State for LINQ Queries:

You may want to start with an empty sequence before applying filtering, projection, or other transformations in LINQ queries. This allows you to build queries conditionally.

```
var query = Enumerable.Empty<Item>()  
    .Where(item => item.SomeCondition)  
    .Select(item => item.Transform());
```

Placeholder for Deferred Execution:

In scenarios where you want to defer execution of a LINQ query until later but need to create a placeholder for the query, `Enumerable.Empty<T>` can be used as a starting point.

```
var deferredQuery = Enumerable.Empty<Item>().Where(item => item.SomeCondition);
```

Empty Sequences for Aggregation:

When aggregating data from multiple sources or needing an initial empty sequence to collect data, you can use `Enumerable.Empty<T>` as a starting point for aggregations.

```
var aggregatedData = sources.Aggregate(Enumerable.Empty<DataType>(), (acc, source) =>  
acc.Concat(source.Data));
```

Default or Fallback Values

Providing a default or fallback value in a conditional expression, particularly in ternary or null-coalescing operations.

Example: `var results = someCondition ? GetData() : Enumerable.Empty<Data>();`

Concatenating Sequences

When you want to concatenate sequences conditionally and need an empty sequence to start with.

Example: Concatenating user permissions where a new user might start with no permissions: `var permissions = Enumerable.Empty<Permission>().Concat(additionalPermissions);`

Avoiding Null Collections

When you want to guarantee that a collection property is never null, to avoid the need for null checks elsewhere in your code.

Example: In a class constructor where collection properties are initialized to `Enumerable.Empty<T>()` rather than left as null.

In summary, `Enumerable.Empty<T>` is valuable when you need to create empty sequences or collections of a specific type for various purposes, such as initialization, testing, handling default values, or serving as a placeholder for data. It allows you to work with empty collections in a type-safe manner and is particularly useful in LINQ queries and deferred execution scenarios.

In the next article, I will discuss the **Linq Append Method in C#** with Examples. In this article, I try to explain the **LINQ Empty Method in C#** with Examples. I hope you enjoy this article.